

SISTEMA DE CONTROL DE FERMENTACIÓN DE CERVEZA

Manual de Instalación y Operativo

ESP32 + Flask Dashboard — Versión Simulada

1. Introducción

Este documento describe los pasos necesarios para instalar, configurar y poner en marcha el sistema de control de fermentación de cerveza. El sistema consta de dos partes:

- `micropython_main.py` — firmware MicroPython que corre en el ESP32. Simula los valores de temperatura y humedad por software, y activa físicamente el relé de enfriamiento (GPIO 25) según la lógica bang-bang. Cuando el relé se activa, enciende en paralelo la bomba de agua, el ventilador y la placa Peltier.
- `flask_app.py` — servidor web Python que corre en el PC. Lanza el firmware en el ESP32 via `mpremote`, lee los datos por puerto serie USB y los expone en un dashboard web local (<http://127.0.0.1:5000>).

2. Hardware

2.1 Lista de componentes

Componente	Descripción
ESP32 DevKit v1	Microcontrolador principal (38 pines, lógica 3.3V)
Módulo relé 5V (Low Level Trigger)	Controla en paralelo la bomba de agua, el ventilador y la placa Peltier
Bomba de agua	Circula el fluido de enfriamiento por el serpentín del fermentador
Ventilador	Disipa el calor del lado caliente de la placa Peltier
Placa Peltier (TEC1-12706 o similar)	Módulo termoelectrónico que enfría el fluido
Sensor DS18B20	Sensor digital de temperatura 1-Wire.
Resistencia 4.7 kΩ	Pull-up entre el pin DATA del DS18B20 y 3.3V
Pantalla TFT ILI9341 (SPI)	Display 320×240 px — muestra temperatura y estado

Cable USB-A a micro-USB	Comunicación serie PC ↔ ESP32 y alimentación del ESP32
-------------------------	--

2.2 Conexiones de hardware

Relé (GPIO 25):

Pin ESP32	Pin módulo relé	Descripción
GPIO 25	IN	Señal de control. LOW = relé activo. Activa bomba, ventilador y Peltier en paralelo.
5V / VIN	VCC	Alimentación del módulo relé
GND	GND	Masa común

Sensor DS18B20:

Pin DS18B20	Conexión en ESP32	Descripción
VCC (pin 3)	3.3V	Alimentación del sensor. Conectar solo a 3.3V.
GND (pin 1)	GND	Masa
DATA (pin 2)	GPIO 4	Línea de datos 1-Wire
DATA (pin 2)	3.3V	Pull-up obligatorio entre DATA y 3.3V

Pantalla TFT ILI9341:

Pin ESP32	Pin TFT	Descripción
GPIO 18	SCK	Reloj SPI
GPIO 23	MOSI / SDI	Datos SPI salida
GPIO 19	MISO / SDO	Datos SPI entrada
GPIO 15	CS	Chip Select
GPIO 2	DC / RS	Data / Command
GPIO 4	RST	Reset pantalla
3.3V	VCC	Alimentación pantalla
GND	GND	Masa

Si no se conecta la TFT, el firmware la detecta automáticamente con try/except y continúa sin ella (USE_TFT = False).

En la versión simulada actual el DS18B20 no se lee por software — la temperatura se genera internamente. El pin GPIO 4 está asignado al RST de la TFT. Para integrar el sensor en producción sería necesario reasignarlo a un GPIO libre y sustituir la lógica de simulación por la lectura 1-Wire.

3. Instalación del software

3.1 Instalar dependencias Python en el PC

```
pip install flask pyserial mpremote
```

3.2 Flashear MicroPython en el ESP32

1. Instalar esptool:

```
pip install esptool
```

2. Descargar la imagen MicroPython para ESP32 desde <https://micropython.org/download/esp32/>
3. Conectar el ESP32 por USB al PC.
4. Borrar la flash del ESP32:

```
esptool.py --chip esp32 --port COM3 erase_flash
```

5. Escribir MicroPython:

```
esptool.py --chip esp32 --port COM3 write_flash -z 0x1000 esp32-vX.Y.Z.bin
```

Sustituir COM3 por el puerto real (Administrador de dispositivos en Windows / dmesg en Linux). El archivo micropython_main.py no necesita copiarse al ESP32 — mpremote lo ejecuta directamente desde el PC.

4. Configuración

4.1 Parámetros del firmware — micropython_main.py

Variable	Valor por defecto	Descripción
temperatura	20.8 °C	Temperatura inicial de la simulación
setpoint	19.5 °C	Temperatura objetivo de fermentación
cool_on_temp	20.1 °C (setpoint + 0.6)	Umbral de activación del relé — enciende bomba, ventilador y Peltier
cool_off_temp	18.9 °C (setpoint - 0.6)	Umbral de desactivación del relé — apaga bomba, ventilador y Peltier
temp_min / temp_max	18.6 / 21.2 °C	Límites del clamp de temperatura simulada

agua_temp	16.2 °C	Temperatura inicial del fluido (simulada)
humedad_pct	62.0 %	Humedad inicial (simulada)
duracion_prueba	120 s	Duración del ciclo completo de fermentación simulada
RELAY_ACTIVE_LOW	True	True = relé se activa con LOW en GPIO 25 (Low Level Trigger)

Variación de valores simulados por iteración (cada 2 segundos):

Estado del relé	Temperatura	Agua	Humedad
ON — bomba + ventilador + Peltier activos	-0.62 °C	-0.28 °C	+0.16 %
OFF — todo apagado	+0.48 °C	+0.14 °C	+0.05 %

4.2 Parámetros del servidor Flask — flask_app.py

Variable de entorno	Por defecto	Descripción
FIRMWARE_PORT	COM3	Puerto serie del ESP32
AUTO_START_FIRMWARE	1	1 = lanza mpremote automáticamente al iniciar Flask

```
set FIRMWARE_PORT=COM5           (Windows CMD)
$env:FIRMWARE_PORT='COM5'        (Windows PowerShell)
export FIRMWARE_PORT=/dev/ttyUSB0 (Linux / Mac)
```

5. Puesta en marcha

5.1 Arranque estándar

6. Conectar el ESP32 al PC por USB.
7. Verificar el puerto serie asignado.
8. Si el puerto no es COM3, definir FIRMWARE_PORT antes de continuar.
9. Lanzar el servidor:

```
python flask_app.py
```

10. Flask lanza automáticamente: mpremote connect COM3 run micropython_main.py
11. El navegador se abre solo en http://127.0.0.1:5000 cuando el puerto está listo.
12. El dashboard se actualiza en tiempo real cuando el ESP32 empieza a enviar JSON_DATA.

5.2 Arranque manual del firmware

Con AUTO_START_FIRMWARE=0, pulsar 'Iniciar Firmware' en el dashboard, o desde terminal:

```
mpremote connect COM3 run micropython_main.py
```

5.3 Flujo de datos

Campo JSON ESP32	Campo Flask state	Descripción
temp	temperature_c	Temperatura simulada del fermentador (°C)
enfriando	pump_on / fan_speed_pct / peltier_power_pct	True = relé ON. Flask refleja bomba, ventilador y Peltier al 100%.
setpoint	setpoint_c	Temperatura objetivo
water_temp	water_temp_c	Temperatura del fluido de enfriamiento (simulada)
humidity_pct	humidity_pct	Humedad relativa (simulada)
beer_progress	beer_progress_pct	Porcentaje completado (0-100)
beer_ready	beer_ready	True cuando elapsed_s >= duracion_prueba
beer_phase	beer_phase	Texto del estado actual
beer_gravity	beer_gravity	Densidad del mosto (1.050 → 1.010)

5.4 Verificación del sistema

Indicador	Estado correcto
Consola Flask al arrancar	"Lanzando puente serial: mpremote connect COMx run micropython_main.py"
Consola Flask en ejecución	Líneas "mpremote: JSON_DATA:{...}" cada ~2 segundos
Dashboard — ESP32	"Conectado" (verde)
Dashboard — temperatura	Oscila entre 18.6 °C y 21.2 °C
Dashboard — progreso	Sube de 0% a 100% en 120 segundos
Relé físico	Click audible cuando temperatura simulada supera 20.1 °C
Fin de ciclo	Consola: "=== PROGRESO 100% ALCANZADO, DETENIENDO MICROPYTHON ==="

6. Endpoints de la API REST

Método	Ruta	Descripción
GET	/	Sirve el dashboard HTML
GET	/api/state	Estado completo del sistema en JSON + historial de 120 lecturas
POST	/api/control	Modifica: mode, setpoint_c, fan_speed_pct, pump_on, peltier_power_pct. Con action=reset reinicia la simulación.
POST	/api/run_firmware	Lanza mpremote si no está ya activo

7. Solución de problemas

Síntoma	Causa probable	Solución
Dashboad muestra 0 en todo	mpremote no conectó al puerto	Verificar FIRMWARE_PORT y que el ESP32 está conectado
"No se detectó COM3 ni COM5"	ESP32 en otro puerto	Definir FIRMWARE_PORT con el puerto correcto
Error de importación en mpremote	MicroPython no flasheado	Repetir paso 3.2
Relé hace click pero no actúa sobre la carga	Lógica de nivel invertida	Cambiar RELAY_ACTIVE_LOW = False en micropython_main.py
DS18B20 no responde / lecturas erróneas	Sin resistencia pull-up o pull-up incorrecto	Verificar resistencia 4.7 kΩ entre DATA y 3.3V
Pantalla TFT no enciende	Driver ili9341 no disponible	No es crítico — firmware continúa con USE_TFT = False automáticamente
Ciclo dura muy poco o muy largo	duracion_prueba mal ajustada	Modificar duracion_prueba en micropython_main.py (en segundos)
El navegador no se abre solo	Flask tardó en arrancar	Abrir manualmente http://127.0.0.1:5000